

LO6 -- Evaluate the Use of Selected Container Orchestration Systems

Based on the uploaded learning outcomes. filecite學turn0file0學L1-L1像

1. Kubernetes networking vs Docker networking

Answer: Kubernetes assigns every Pod its own IP address and follows a flat networking model where every Pod can communicate with every other Pod without NAT. Services provide stable virtual IPs and DNS-based discovery. Docker networking is host-centric, typically using bridge networks with NAT and port publishing.

Evaluation: Kubernetes networking is more scalable and cloud-native, while Docker networking is simpler and better suited for single-host deployments.

2. Kubernetes storage vs Podman storage

Kubernetes uses CSI, StorageClasses and dynamic provisioning, allowing automatic creation of storage from many vendors. Podman mainly relies on local volumes and storage plugins on one host.

Conclusion: Kubernetes is significantly more flexible for stateful workloads because storage is portable, dynamic and cluster-aware.

3. Operator pattern vs plain manifests

Operators combine CRDs with controllers to automate deployment, upgrades, backups and recovery of complex applications.

Advantages - Self-healing - Automated upgrades - Backup automation - Less manual administration

Disadvantages - More complexity - Requires operator knowledge

For databases and stateful services, Operators are preferred.

4. Kubernetes vs OpenShift S2I

Plain Kubernetes focuses on flexibility using kubectl, YAML and CI/CD tools. OpenShift adds Source-to-Image (S2I), Developer Console and integrated pipelines.

Evaluation - Kubernetes optimizes flexibility. - OpenShift optimizes developer productivity.

5. Migrating OpenShift → Kubernetes

Benefits: - Less vendor lock-in - Lower licensing cost - Wider community support

Challenges: - Replace OpenShift Routes with Ingress. - Replace SCC with Pod Security. - Replace integrated registry/build tools. - Reconfigure authentication and CI/CD.

Migration is feasible but requires planning.

6. Build agents in Kubernetes vs VMs

Kubernetes: - Ephemeral - Auto-scaled - Better isolation - Lower idle cost

VMs: - Easier to troubleshoot - Stable environment - Higher maintenance

Recommendation: Kubernetes for modern CI/CD.

7. Cloud integration

Kubernetes integrates with cloud providers through: - CSI - CNI - Cloud Controller Manager - Cluster Autoscaler - Load Balancers - Managed disks - Managed databases

Resources are provisioned dynamically as demand increases.

8. OpenShift security vs Kubernetes

OpenShift provides: - Secure defaults - SELinux - Restricted Security Context Constraints - Integrated image scanning - OAuth - Compliance tools

Vanilla Kubernetes requires manual hardening.

Regulated industries often choose OpenShift because security is enabled by default.

9. Learning curve

Kubernetes requires deeper knowledge of YAML, networking and troubleshooting.

OpenShift adds abstractions that simplify development but introduces platform-specific concepts.

For beginners OpenShift is often easier operationally.

10. Kubernetes vs k3s

k3s: - Lightweight - Low RAM usage - Single binary - Edge/IoT/Home labs

Full Kubernetes: - Enterprise features - Large clusters - HA control plane

Full Kubernetes is overkill for small deployments.

11. Compose vs Kubernetes

Use Docker Compose when: - One server - Small team - Simple deployment

Use Kubernetes when: - High availability - Scaling - Multiple hosts - Self-healing

Recommendation: Start with Compose unless growth requires orchestration.

12. Vendor lock-in

Kubernetes is CNCF open source and portable. OpenShift adds Red Hat tooling and support but introduces some vendor dependence.

13. Why recommend OpenShift

- Vendor support
- Integrated registry
- CI/CD
- Monitoring
- Logging
- Security
- Enterprise support

Ideal for large organizations.

14. Storage provisioning

VMs require administrators to manually create disks. Kubernetes provisions storage automatically through StorageClasses and CSI.

Kubernetes provides greater automation.

15. Startup recommendation

Recommend Docker Compose or Podman Compose.

Reasons: - Lowest cost - Minimal operational overhead - Fast deployment - Easy learning curve

Migrate to Kubernetes later.

16. Docker vs Podman

Docker: - Daemon architecture - Root daemon - Larger attack surface

Podman: - Daemonless - Rootless support - OCI compatible - Better security

Security-conscious organizations often prefer Podman.

17. Self-managed vs Managed Kubernetes

Self-managed: - Full control - Responsible for upgrades, backups and HA

Managed: - Cloud provider manages control plane - Easier maintenance - Faster updates - Higher service cost

Managed Kubernetes greatly reduces operational effort.

18. Media streaming

Recommend managed Kubernetes with autoscaling and global load balancing.

Reasons: - Horizontal Pod Autoscaler - Cluster Autoscaler - CDN integration - High availability - Multi-region deployment

19. Docker Swarm → Kubernetes

Migration requires effort but provides: - Better scaling - Self-healing - Larger ecosystem - Better networking - Better storage - Long-term support

Worth migrating for growing companies.

20. Kubernetes for microservices

Recommendation: Yes.

Advantages: - Service discovery - Load balancing - Rolling updates - Self-healing - Autoscaling - Huge ecosystem

21. Kubernetes for enterprise

Recommendation: Yes.

Reasons: - Massive scalability - Mature ecosystem - Community support - Security features - Cloud portability - Automation

22. OpenShift for microservices

Recommendation: Yes.

Adds: - Enterprise security - Developer tools - Pipelines - Monitoring - Logging - Operators

Excellent for regulated organizations.

23. OpenShift for enterprise

Recommendation: Strongly yes.

Benefits: - Vendor support - Enterprise security - Integrated CI/CD - Compliance - Monitoring
- Automation - Excellent scalability

Exam tip: When asked to *evaluate* or *justify*, always include both advantages and disadvantages before giving your recommendation.